

目录

一、宏和全局变量	1
二、MIPS 虚存映射布局	19
三、记录内存与页表的数据结构分析	20
四、内存初始化	22
五、内存分配与释放	25
六、地址转换与 TLB 重填	30
七、附加题：建表与映射函数	34

一、宏和全局变量

链表管理宏

LIST_HEAD 原分析报告仅提到“创建链表头”，实际上该宏会声明一个包含 `lh_first` 指针的结构体类型，用于管理首结点。需要和 `LIST_HEAD_INITIALIZER` 搭配使用。

`include/queue.h` (5-8)

```
5  #define LIST_HEAD(name, type)
   ↪ \
6      struct name {
   ↪ \
7          struct type *lh_first; /* 指向第一个元素 */
   ↪ \
8      }
```

LIST_HEAD_INITIALIZER(更正原分析报告中的拼写) 原文给出的 `LIST_HEAD_INITIALIZE` 拼写有误，正确宏会把头指针置空，既可用于静态初始化也可用于链表重置。

`include/queue.h` (10-10)

```
10 #define LIST_HEAD_INITIALIZER(head) { NULL } // 头结点初始为空
```

LIST_ENTRY 并非“链表第一个节点”，而是定义链表节点内部的 `le_next/le_prev` 指针字段，供任意结构体嵌入。

`include/queue.h` (12-16)

```
12 #define LIST_ENTRY(type)
   ↪ \
13     struct {
   ↪ \
14         struct type *le_next; /* 后继结点 */
   ↪ \
```

```

15 |         struct type **le_prev; /* 指向前驱 next 指针的指针
    |           ↪ */ \
16 |     }

```

LIST_EMPTY 用 lh_first == NULL 判断链表是否为空，需传入表头指针。

include/queue.h (18-18)

```

18 | #define LIST_EMPTY(head)      ((head)->lh_first == NULL) // 判空

```

LIST_FIRST 返回首元指针，是对 head→lh_first 的安全包装。

include/queue.h (19-19)

```

19 | #define LIST_FIRST(head)      ((head)->lh_first)          //
    |     ↪ 取首元

```

LIST_FOREACH 原分析报告未提及循环条件，该宏以 LIST_FIRST 为起点，以 LIST_NEXT 为推进，保证遍历整个链表。

include/queue.h (21-24)

```

21 | #define LIST_FOREACH(var, head, field)
    |     ↪ \
22 |         for ((var) = LIST_FIRST((head)); /* 从首元开始 */
    |           ↪ \
23 |             (var); /* 未到尾部 */
    |           ↪ \
24 |             (var) = LIST_NEXT((var), field)) /* 迭代到下一个 */

```

LIST_INIT 执行时会把 lh_first 置空，常用于运行期重置链表。原分析报告把它描述成“创建新链表”略显含糊。

include/queue.h (26-28)

```

26 #define LIST_INIT(head) do {                                \
27     LIST_FIRST((head)) = NULL;                                \
28     } while (0) // 将链表置空

```

LIST_INSERT_AFTER 并非只修改“新节点两个指针”，实现上同时维护被插入节点原后继元素的 `le_prev`，以保证双向一致。

include/queue.h (32-39)

```

32 #define LIST_INSERT_AFTER(listelm, elm, field) do {
    ↪ \
33     LIST_NEXT((elm), field) = LIST_NEXT((listelm),
    ↪ field); \
34     if (LIST_NEXT((listelm), field) != NULL)
    ↪ \
35         LIST_NEXT((listelm), field)->field.le_prev
    ↪ = \
36         &LIST_NEXT((elm), field);
    ↪ \
37     LIST_NEXT((listelm), field) = (elm);
    ↪ \
38     (elm)->field.le_prev = &LIST_NEXT((listelm),
    ↪ field); \
39 } while (0) // 在指定元素之后插入

```

LIST_INSERT_BEFORE (纠正原分析报告中“LIST_INSERT_TAIL 在前插入”的写法)
原分析报告将“在某节点前插入”误写成 `LIST_INSERT_TAIL`。真正执行前插的是 `LIST_INSERT_BEFORE`，其核心是重定向待插节点和原节点的前驱指针。

include/queue.h (41-46)

```

41 #define LIST_INSERT_BEFORE(listelm, elm, field) do {
    ↪ \
42         (elm)->field.le_prev = (listelm)->field.le_prev;
    ↪ \
43         LIST_NEXT((elm), field) = (listelm);
    ↪ \
44         *(listelm)->field.le_prev = (elm);
    ↪ \
45         (listelm)->field.le_prev = &LIST_NEXT((elm),
    ↪ field); \
46     } while (0) // 在指定元素之前插入

```

LIST_INSERT_HEAD 除非链表为空，否则需要同时修正原首元的 `le_prev`，确保双向链正确。原分析报告的描述缺少这一点。

include/queue.h (48-53)

```

48 #define LIST_INSERT_HEAD(head, elm, field) do {
    ↪ \
49         if ((LIST_NEXT((elm), field) = LIST_FIRST((head)))
    ↪ != NULL) \
50             LIST_FIRST((head))->field.le_prev =
    ↪ &LIST_NEXT((elm), field); \
51         LIST_FIRST((head)) = (elm);
    ↪ \
52         (elm)->field.le_prev = &LIST_FIRST((head));
    ↪ \
53     } while (0) // 头插

```

LIST_INSERT_TAIL 内核源码提供了线性扫描尾插版本，会在链表为空时退化为头插，和原分析报告标注的“仅需前后指针”不同。

include/queue.h (55-69)

```

55 #define LIST_INSERT_TAIL(head, elm, field) do {
    ↪ \
56     if (LIST_FIRST((head)) == NULL) {
        ↪ \
57         LIST_FIRST((head)) = (elm);
        ↪ \
58         (elm)->field.le_prev =
        ↪ &LIST_FIRST((head)); \
59     } else {
        ↪ \
60         LIST_NEXT((elm), field) =
        ↪ LIST_FIRST((head)); \
61         while (LIST_NEXT((LIST_NEXT((elm),
        ↪ field)), field)) { \
62             LIST_NEXT((elm), field) =
        ↪ \
63                 LIST_NEXT(LIST_NEXT((elm),
        ↪ field), field); \
64         }
        ↪ \
65         (LIST_NEXT((LIST_NEXT((elm), field)),
        ↪ field)) = (elm); \
66         (elm)->field.le_prev = &LIST_NEXT((elm),
        ↪ field); \
67         (elm)->field.le_next = NULL;
        ↪ \
68     }
    ↪ \
69 } while (0) // 尾插 (线性查找尾部)

```

LIST_REMOVE 删除节点时既要更新前驱保存的 `le_prev`，又要在存在后继时修正它的 `le_prev`，否则链表会断裂。

include/queue.h (71-76)

```

71 | #define LIST_REMOVE(elm, field) do {
    | ↪ \
72 |         if (LIST_NEXT((elm), field) != NULL)
    | ↪ \
73 |             LIST_NEXT((elm), field)->field.le_prev =
    | ↪ \
74 |                 (elm)->field.le_prev;
    | ↪ \
75 |             *(elm)->field.le_prev = LIST_NEXT((elm), field);
    | ↪ \
76 |     } while (0) // 删除指定元素

```

页号与地址转换宏

BY2PG 实验指导只说“定义页大小”，源码明确固定为 4096 字节，对应 4KB 页框，是所有对齐逻辑的基准。

include/mmu.h (5-5)

```

5 | #define BY2PG    4096           // 每页 4KB

```

PDMAP 该常量等于 4MB，指出单个页目录项（PDE）可映射的跨度，原分析报告未给出数值。

include/mmu.h (6-6)

```

6 | #define PDMAP    (4*1024*1024) // 一个页目录项覆盖 4MB

```

PGSHIFT 确认为 12，对应 2 的 12 次方即 4KB。用于页号与地址之间左/右移转换。

include/mmu.h (7-7)

```

7 | #define PGSHIFT  12           // 页内偏移位数 (log2(BY2PG))

```

PDX(va) 不仅“提取页目录索引”，还屏蔽出虚拟地址的高 10 位，供一级页表索引使用。

include/mmu.h (11-11)

```
11 | #define PDX(va) (((u_long)(va)) >> 22) & 0x03FF) // 页目录索引 (高  
    | ↪ 10 位)
```

PTX(va) 与 PDX 类似，负责提取中间 10 位以索引二级页表。原分析报告重复列出一
次，可合并理解。

include/mmu.h (12-12)

```
12 | #define PTX(va) (((u_long)(va)) >> 12) & 0x03FF) // 页表索引 (中 10  
    | ↪ 位)
```

PTE_ADDR(pte) 原分析报告把参数写成“va”略显不准。宏通过屏蔽低 12 位得到某个
页表项指向的物理页基址。

include/mmu.h (14-14)

```
14 | #define PTE_ADDR(pte) ((u_long)(pte) & ~0xFFF) // PTE  
    | ↪ 对应的物理页框基址
```

PPN(x) 原文说“从物理地址提取页号”是准确的，补充说明可以同时处理虚拟地址或
物理地址，效果都是截断低 12 位。

include/mmu.h (17-18)

```
17 | #define PPN(va) (((u_long)(va)) >> 12) // 物理/虚拟页号  
18 | #define VPN(va) PPN(va) // 同义宏：虚拟页号
```

VA2PFN(va) 它不是直接得到“页框号”整数，而是把低 12 位清零后返回可写入 CP0
EntryLo 的 PFN 字段。

include/mmu.h (21-21)

```
21 #define VA2PFN(va) (((u_long)(va)) & 0xFFFFF000) // 清零低 12 位得到
    ↪ PFN
```

PTE_V 有效位在 0x0200, 硬件据此判断 PTE/PDE 是否可用; 缺失会触发 TLB 异常。

include/mmu.h (25-27)

```
25 #define PTE_G      0x0100 // 全局页
26 #define PTE_V      0x0200 // 有效
27 #define PTE_R      0x0400 // 可写/脏 (0 表示只读)
```

PTE_R 在 MIPS 本实验里承担“是否允许写入”与“脏位”双重角色, 原分析报告仅说明“可读”需要修正。

include/mmu.h (25-27)

```
25 #define PTE_G      0x0100 // 全局页
26 #define PTE_V      0x0200 // 有效
27 #define PTE_R      0x0400 // 可写/脏 (0 表示只读)
```

ULIM/UVPT/UPAGES/UENVS 这组常量共同描述高地址布局: ULIM 分隔内核与用户空间; 往下依次为 UVPT、UPAGES、UENVS, 每段占用 PDMAP 大小。

include/mmu.h (34-42)

```
34 #define KERNBASE 0x80010000 // 内核镜像基址 (链接脚本一致)
35 #define VPT      (ULIM + PDMAP) // 内核页表自映射窗口
36 #define KSTACKTOP (VPT - 0x100) // 内核栈顶 (向下增长)
37 #define KSTKSIZE  (8*BY2PG)    // 内核栈大小: 8 页
38 #define ULIM      0x80000000    // 内核高半区起始
39
40 #define UVPT      (ULIM - PDMAP) // 用户只读自映射页表窗口
```

```

41 #define UPAGES (UVPT - PDMAP) // 物理页描述数组 (struct Page[])
42 #define UENVS (UPAGES - PDMAP) // 进程控制块数组

```

PDMAP (再次说明用途) 配合前述常量, PDMAP 让 UVPT/UPAGES/UENVS 以整块只读窗口映射。

include/mmu.h (5-42)

```

5 #define BY2PG 4096 // 每页 4KB
6 #define PDMAP (4*1024*1024) // 一个页目录项覆盖 4MB
7 #define PGSHIFT 12 // 页内偏移位数 (log2(BY2PG))
8 #define PDSHIFT 22 // 目录对齐位数 (log2(PDMAP))
9
10 // 从虚拟地址中提取索引
11 #define PDX(va) (((u_long)(va)) >> 22) & 0x03FF // 页目录索引 (高
    ↳ 10 位)
12 #define PTX(va) (((u_long)(va)) >> 12) & 0x03FF // 页表索引 (中 10
    ↳ 位)
13
14 #define PTE_ADDR(pte) ((u_long)(pte) & ~0xFFF) // PTE
    ↳ 对应的物理页框基址
15
16 // 页号相关 (去除页内偏移)
17 #define PPN(va) (((u_long)(va)) >> 12) // 物理/虚拟页号
18 #define VPN(va) PPN(va) // 同义宏: 虚拟页号
19
20 // 供 TLB EntryLo 使用的 PFN
21 #define VA2PFN(va) (((u_long)(va)) & 0xFFFFF000) // 清零低 12 位得到
    ↳ PFN
22 #define PTE2PT 1024 // 每个目录项对应
    ↳ 1024 个 PTE
23
24 // PTE 标志位 (硬件/约定)
25 #define PTE_G 0x0100 // 全局页
26 #define PTE_V 0x0200 // 有效
27 #define PTE_R 0x0400 // 可写/脏 (0 表示只读)

```

```

28 #define PTE_D      0x0002 // 文件缓存脏标记
29 #define PTE_COW    0x0001 // 写时复制
30 #define PTE_UC      0x0800 // 不经缓存
31 #define PTE_LIBRARY 0x0004 // 共享页
32
33 // 内核与用户关键地址常量
34 #define KERNBASE 0x80010000 // 内核镜像基址 (链接脚本一致)
35 #define VPT      (ULIM + PDMAP) // 内核页表自映射窗口
36 #define KSTACKTOP (VPT - 0x100) // 内核栈顶 (向下增长)
37 #define KSTKSIZE  (8*BY2PG)     // 内核栈大小: 8 页
38 #define ULIM      0x80000000     // 内核高半区起始
39
40 #define UVPT      (ULIM - PDMAP) // 用户只读自映射页表窗口
41 #define UPAGES    (UVPT - PDMAP) // 物理页描述数组 (struct Page[])
42 #define UENVS     (UPAGES - PDMAP) // 进程控制块数组

```

PADDR(kva) 需要保证参数位于内核高半区 ($\geq$ ULIM), 否则会 panic; 原分析报告中 “PADDR(freemem)” 只是特例。

include/mmu.h (82-88)

```

82 #define PADDR(kva) \
83 ({ \
84     u_long a = (u_long)(kva); \
85     if (a < ULIM) \
86         panic("PADDR called with invalid kva %08lx", a); \
87     a - ULIM; \
88 })

```

KADDR(pa) 以 $PPN(pa) < npage$ 为界限, 把物理地址投射到 ULIM 以上的线性映射区。

include/mmu.h (91-97)

```

91 #define KADDR(pa) \
92 ({ \
93     u_long ppn = PPN(pa); \
94     if (ppn >= npage) \
95         panic("KADDR called with invalid pa %08lx", (u_long)pa); \
96     (pa) + ULIM; \
97 })

```

PDE/Pte 类型 源码中用 `typedef u_long` 明确页表项宽度 32 位，补上原分析报告未写出的类型声明。

include/mmu.h (75-77)

```

75 typedef u_long Pde; // 页目录项
76 typedef u_long Pte; // 页表项
77

```

VA2PA 辅助函数 原分析报告将 `va2pa` 归入“宏”，但它其实是内联函数，会先检查 PDE/PTE 的有效位再返回物理地址。

include/pmap.h (38-47)

```

38 static inline u_long va2pa(Pde* pgdir, u_long va) {
39     Pte* p;
40     pgdir = &pgdir[PDX(va)];
41     if (!(*pgdir & PTE_V))
42         return ~0;
43     p = (Pte*)KADDR(PTE_ADDR(*pgdir));
44     if (!(p[PTX(va)] & PTE_V))
45         return ~0;
46     return PTE_ADDR(p[PTX(va)]);
47 }

```

ROUND 定义位于 `types.h`，通过“加 $n-1$ 再清零低位”实现向上对齐，要求 n 为 2 的幂。

include/types.h (36-37)

```

36 #define ROUND(a, n)      (((((u_long)(a)) + (n) - 1)) & ~((n) - 1))
    ↪ // 向上按 n 对齐 (n 为 2 的幂)
37 #define ROUNDDOWN(a, n) (((u_long)(a)) & ~((n) - 1))
    ↪ // 向下按 n 对齐 (n 为 2 的幂)

```

panic **panic** 是 **_panic** 的包装，会自动携带文件名与行号；原分析报告可以补上这一点。

include/printf.h (7-10)

```

7 void _panic(const char*, int, const char*, ...) //
    ↪ 内核致命错误输出并停机
8     __attribute__((noreturn));                  // 函数不返回
9
10 #define panic(...) _panic(__FILE__, __LINE__, __VA_ARGS__) //
    ↪ 带文件与行号的 panic 宏

```

npage 并非宏，而是 **u_long** 全局变量，由 **mips_detect_memory()** 设定为物理页总数。

mm/pmap.c (8-17)

```

8 /* 以下变量由 mips_detect_memory() 初始化，描述物理内存规模与范围 */
9 /* 由 mips_detect_memory() 进行赋值 */
10 u_long maxpa;           /* 物理地址上限 */
11 u_long npage;           /* 物理页数量 (以页为单位) */
12 u_long basemem;         /* 基础内存大小 (字节) */
13 u_long extmem;          /* 扩展内存大小 (字节) */
14
15 Pde *boot_pgdir; // 中文注释
16
17 struct Page *pages; // 中文注释

```

BY2PG 与 PGSHIFT 的联动 在原分析报告中分别出现，源码表明用 PGSHIFT 做位移可以从页号计算回 BY2PG 对齐地址。

include/mmu.h (5-8)

```
5 #define BY2PG    4096           // 每页 4KB
6 #define PDMAP    (4*1024*1024) // 一个页目录项覆盖 4MB
7 #define PGSHIFT  12           // 页内偏移位数 (log2(BY2PG))
8 #define PDSHIFT  22           // 目录对齐位数 (log2(PDMAP))
```

ULIM 再说明 ULIM 也被 PADDR/KADDR 用作合法性判断的阈值，是内核线性映射的起点。

include/mmu.h (34-38)

```
34 #define KERNBASE 0x80010000 // 内核镜像基址 (链接脚本一致)
35 #define VPT      (ULIM + PDMAP) // 内核页表自映射窗口
36 #define KSTACKTOP (VPT - 0x100) // 内核栈顶 (向下增长)
37 #define KSTKSIZE  (8*BY2PG)     // 内核栈大小: 8 页
38 #define ULIM      0x80000000     // 内核高半区起始
```

NENV 通过 LOG2NENV 控制大小，当前设为 1024；与 ENVX、GET_ENV_ASID 等宏共同管理环境索引。

include/env.h (10-14)

```
10 #define LOG2NENV 10           // 环境数量的对数 (以 2 为底)
11 #define NENV     (1 << LOG2NENV) // 最大环境数量
12 #define ENVX(envid) ((envid) & (NENV - 1)) // 从 envid 取索引
13 #define GET_ENV_ASID(envid) (((envid) >> 11) << 6) // 生成 MIPS
    ↪ ASID (示意)
14
```

UPAGES/UENVS 除了虚拟地址常量，还对应着 mips_vm_init 中的两次 boot_map_segment，确保 pages[] 与 envs[] 可在只读窗口访问。

include/mmu.h (40-42)

```

40 #define UVPT    (ULIM - PDMAP) // 用户只读自映射页表窗口
41 #define UPAGES (UVPT - PDMAP) // 物理页描述数组 (struct Page[])
42 #define UENVS  (UPAGES - PDMAP) // 进程控制块数组

```

VA2PFN 与 GET_ENV_ASID 的协同 当 TLB 例程调用 `tlb_out` 或写 `EntryLo` 时，会分别用 VA2PFN 提供 PFN，用 GET_ENV_ASID 区分地址空间。

include/env.h (10-14)

```

10 #define LOG2NENV 10 // 环境数量的对数 (以 2 为底)
11 #define NENV      (1 << LOG2NENV) // 最大环境数量
12 #define ENVX(envid) ((envid) & (NENV - 1)) // 从 envid 取索引
13 #define GET_ENV_ASID(envid) (((envid) >> 11) << 6) // 生成 MIPS
    ↪ ASID (示意)
14

```

page2pa/page2kva/pa2page 这三者不是宏但在原分析报告中被列出。`page2pa` 左移页号得到物理地址；`page2kva` 进一步通过 `KADDR` 获取线性地址；`pa2page` 则包含越界检查。

include/pmap.h (21-36)

```

21 // Page* -> 物理页号 (相对 pages[] 基址的偏移)
22 static inline u_long page2ppn(struct Page* pp) { return pp -
    ↪ pages; }
23
24 // Page* -> 物理地址 (页号左移 12 得到页框基址)
25 static inline u_long page2pa(struct Page* pp) { return
    ↪ page2ppn(pp) << PGSHIFT; }
26
27 // 物理地址 -> Page* (越界检查: PPN(pa) 必须小于 npage)
28 static inline struct Page* pa2page(u_long pa) {

```

```

29     if (PPN(pa) >= npage)
30         panic("pa2page called with invalid pa: %x", pa);
31     return &pages[PPN(pa)];
32 }
33
34 // Page* -> 可访问的内核虚拟地址 (通过 KADDR 恒等映射)
35 static inline u_long page2kva(struct Page* pp) { return
    ↪ KADDR(page2pa(pp)); }
36

```

tlb_out TLB 失效不是“宏”而是 `mm/tlb_asm.S` 中的汇编函数, 接受完整 `EntryHi` (含 `ASID+VPN`) 并执行 `tlbp/tlbwi` 覆写。

`mm/tlb_asm.S (5-34)`

```

5 // 按当前 ASID/虚拟页选择性失效并清空对应 TLB 项。
6 LEAF(tlb_out)                # 进入函数 (宏定义, 设置入口)
7 //1: j 1b                    // 调试占位: 自旋; 保持注释
8 nop                          # 延迟槽空操作, 避免相关冒险
9     mfc0    k1, CP0_ENTRYHI    # 备份原 EntryHi 到 k1 (保存 ASID/VA
    ↪ 高位)
10    mtc0    a0, CP0_ENTRYHI    # 将参数 a0 写入 EntryHi (目标
    ↪ ASID/VA)
11    nop                          # CP0 写后插入 NOP, 等待生效
12                                # 下面执行 tlbp 探测并可能 tlbwi 覆写
13    # 根据 EntryHi 在 TLB 中探测匹配项 (索引写入 CP0_INDEX)
14    tlbp                      # TLB probe: 按 EntryHi 查找并设置 INDEX
15    nop                          # 探测后插入 NOP, 避免相关冲突
16    nop
17    nop
18    nop
19    mfc0    k0, CP0_INDEX      # 读取探测到的索引 INDEX 至 k0
20    # 若未命中 (INDEX 为负), 跳转 NOFOUND
21    bltz    k0, NOFOUND        # 未命中则无需清除, 直接收尾
22    nop                          # 延迟槽 NOP
23    # 命中: 构造空条目写回以实现失效

```

```
mtc0      zero,CP0_ENTRYHI      # 清空 EntryHi (匹配键)
mtc0      zero,CP0_ENTRYLO0     # 清空低半页 (EntryLo0)
nop                               # CP0 写后 NOP
# 按索引写回覆盖 (wipe)
tlbwi                               # TLB write indexed: 用空条目覆盖
NOFOUND:                               # 未命中路径标签

mtc0      k1,CP0_ENTRYHI        # 恢复原 EntryHi (恢复 ASID/VA)

j      ra                               # 返回调用者
nop                               # 延迟槽 NOP
```

VA2PFN 与 pageout 的关系 pageout() 将 VA2PFN(va) 作为页对齐地址插入页表, 因此该宏实际返回带齐整偏移的虚拟地址。

mm/pmap.c (596-621)

[illegible]

```

615     } // 中文注释
616
617     p->pp_ref++; // 中文注释
618
619     page_insert((Pde *)context, p, VA2PFN(va), PTE_R); // 中文注释
620     printf("pageout:\t@@@_0x%x_@@@ ins a page \n", va); //
        ↪ 中文注释
621 } // 中文注释

```

GET_ENV_ASID 生成 ASID 时右移 11 位再左移 6 位，对应 MIPS 8-bit ASID 字段。原分析报告仅提到“获取 ASID”并未说明位宽。

include/env.h (10-14)

```

10 #define LOG2NENV 10 // 环境数量的对数（以 2 为底）
11 #define NENV (1 << LOG2NENV) // 最大环境数量
12 #define ENVX(envid) ((envid) & (NENV - 1)) // 从 envid 取索引
13 #define GET_ENV_ASID(envid) (((envid) >> 11) << 6) // 生成 MIPS
        ↪ ASID（示意）
14

```

PTE_ADDR再强调 在 `tlb_invalidate` 等路径中，需要把 `va` 清掉低 12 位再与 ASID 组合，这也是宏被频繁使用的原因。

mm/pmap.c (389-394)

```

389 { // 中文注释
390     if (curenv) { // 中文注释
391         tlb_out(PTE_ADDR(va) | GET_ENV_ASID(curenv->env_id)); //
            ↪ 中文注释
392     } else { // 中文注释
393         tlb_out(PTE_ADDR(va)); // 中文注释
394     } // 中文注释

```

curenv 该全局指针定义在 `lib/env.c`, 初值为 `NULL`。原分析报告只写“指向当前环境”但未指出存放位置。

`lib/env.c (13-13)`

```
13 struct Env *curenv = NULL;           // the current env
```

全局变量

`maxpa / basemem / extmem / npage` `mips_detect_memory()` 会确定物理地址上界并推导页数, 这四个 `u_long` 变量集中声明在 `mm/pmap.c` 文件头部。

`mm/pmap.c (8-13)`

```
8 /* 以下变量由 mips_detect_memory() 初始化, 描述物理内存规模与范围 */
9 /* 由 mips_detect_memory() 进行赋值 */
10 u_long maxpa;           /* 物理地址上限 */
11 u_long npage;           /* 物理页数量 (以页为单位) */
12 u_long basemem;         /* 基础内存大小 (字节) */
13 u_long extmem;          /* 扩展内存大小 (字节) */
```

boot_pgdir 页目录全局指针在同一文件里定义, 指向内核自用的启动页目录, 供 `pgdir_walk`、`page_insert` 等处引用。

`mm/pmap.c (15-15)`

```
15 Pde *boot_pgdir; // 中文注释
```

pages 数组 真实的 `struct Page *pages` 定义也在 `mm/pmap.c`, 并由 `mips_vm_init()` 分配空间后指向全局描述符数组; 原分析报告中重复列举可合并。

`mm/pmap.c (17-17)`

```
17 struct Page *pages; // 中文注释
```

page_free_list 虽然原分析报告未强调，该 **Page_list** 变量与链表宏配套，负责串联所有空闲页，是理解 **page_alloc/page_free** 的关键。

mm/pmap.c (20-20)

```
20 static struct Page_list page_free_list;    /* 物理页空闲链表 */
```

curenv (再次归类为全局变量) 除了在宏段提及，这里补充定义位于 **lib/env.c**，初始化为 **NULL**，只有在 **env_run** 成功切换后才指向有效环境。

lib/env.c (13-13)

```
13 struct Env *curenv = NULL;                // the current env
```

二、MIPS 虚存映射布局

结合实验二指导书的布局图与内核源码内 **include/mmu.h** 的常量，可将整个 32 位虚拟空间概括如下：

1. **0x0000_0000** 到 **UTEXT** (缺省 **0x0040_0000**) 供用户文本使用，再向上是数据、堆与用户普通栈 (**USTACKTOP/UTOP - 2*BY2PG**)；
2. **UXSTACKTOP=UTOP** 保留用户异常栈，便于用户态页故障处理；
3. **UENVS**、**UPAGES**、**UVPT** 三个窗口每个占 **PDMAP=4MB**，分别映射 **envs[]**、**pages[]** 与完整页表，统一只读；
4. **ULIM=0x8000_0000** 以上属于内核直接映射区，又在最顶端通过 **VPT** 自映射形成页表调试窗口，同时 **KSTACKTOP** 往下分配内核栈；
5. **KERNBASE** 定位内核 ELF 镜像装载地址，确保与硬件 **reset vector** 匹配。

include/mmu.h (34-49)

```
34 #define KERNBASE 0x80010000 // 内核镜像基址 (链接脚本一致)
35 #define VPT      (ULIM + PDMAP) // 内核页表自映射窗口
36 #define KSTACKTOP (VPT - 0x100) // 内核栈顶 (向下增长)
37 #define KSTKSIZE  (8*BY2PG)    // 内核栈大小: 8 页
38 #define ULIM      0x80000000    // 内核高半区起始
39
40 #define UVPT      (ULIM - PDMAP) // 用户只读自映射页表窗口
```

```

41 #define UPAGES (UVPT - PDMAP) // 物理页描述数组 (struct Page[])
42 #define UENVS (UPAGES - PDMAP) // 进程控制块数组
43
44 #define UTOP      UENVS          // 用户地址空间上界
45 #define UXSTACKTOP (UTOP)        // 用户异常栈顶
46 #define TIMESTACK 0x82000000    // 计时器中断栈 (特殊用途)
47
48 #define USTACKTOP (UTOP - 2*BY2PG) // 普通用户栈顶
49 #define UTEXT     0x00400000     // 用户程序文本起始地址

```

三、记录内存与页表的数据结构分析

1. Page_list 及 Page_LIST_entry_t

```

include/pmap.h (10-12)

10 LIST_HEAD(Page_list, Page);          // 页链表头类型
11 typedef LIST_ENTRY(Page) Page_LIST_entry_t; // 每个 Page
    ↳ 的链表指针字段类型
12

```

这段定义把 BSD queue 的 LIST 头与条目别名化：Page_list 是空闲链表的头部类型，Page_LIST_entry_t 是可嵌入 struct Page 的链表字段。依靠 LIST_HEAD、LIST_ENTRY 等宏，分页分配器得以用 O(1) 插入/删除维护空闲页集合。

2. struct Page

```

include/pmap.h (14-17)

14 struct Page { // 物理页描述符
15     Page_LIST_entry_t pp_link; // 空闲链表指针
16     u_short pp_ref;           // 被页表引用的次数 (仅运行期分配的页有效)
    ↳
17 }; // 结构体结束

```

每个物理页对应一个 struct Page: pp_link 嵌入上述链表; pp_ref 记录被多少页表项引用 (仅运行期动态映射会递增)。page_init 会遍历 pages[] 初始化这两个字段, page_alloc/page_free 基于它们做状态转换。

3. struct Env

include/env.h (19-41)

```

19 struct Env { // 进程/环境控制块
20     struct Trapframe env_tf;           // 最近一次陷入时的现场
21     LIST_ENTRY(Env) env_link;          // 全局空闲/就绪链表链接
22     u_int env_id;                      // 唯一标识符
23     u_int env_parent_id;               // 父环境 ID
24     u_int env_status;                  // 运行状态
25     Pde* env_pgdir;                   // 页目录基址
26     u_int env_cr3;                    // CR3/页表基物理地址或等价字段
27     LIST_ENTRY(Env) env_sched_link;    // 调度队列链接
28     u_int env_pri;                    // 调度优先级
29
30     u_int env_ipc_value;               // IPC 传递的值
31     u_int env_ipc_from;               // IPC 发送方 env_id
32     u_int env_ipc_recving;            // 是否正在接收
33     u_int env_ipc_dstva;              // IPC 映射目标 VA
34     u_int env_ipc_perm;               // IPC 映射权限
35
36     u_int env_pgfault_handler;         // 用户态缺页处理函数入口
37     u_int env_xstacktop;              // 用户异常栈顶
38
39     u_int env_runs;                   // 被调度运行的次数
40     u_int env_nop;                   // 预留字段
41 }; // 结构体结束

```

环境控制块同时保存 Trapframe、调度信息与 env_pgdir (页目录基址)。因此它既参与“记录当前地址空间”的逻辑 (curenv→env_pgdir)，也在 IPC/缺页处理时提供用户栈、异常处理入口等上下文，直接决定页表切换行为。

4. Pde/Pte 类型

include/mmu.h (75-77)

```

75 typedef u_long Pde; // 页目录项
76 typedef u_long Pte; // 页表项

```

虽然只是 `typedef u_long`，但它们统一了页目录/页表项的表示，保证 `pgdir_walk`、`page_insert` 等函数在 C 层拿到的都是 32 位值。结合 `PTE_ADDR/PTE_V` 等宏即可解码/设置页表项内容，是链接软硬件地址空间的基石。

四、内存初始化

1. `mips_detect_memory()`

`mm/pmap.c (23-39)`

```
23  /* 概览：初始化 basemem 与 npage。
24   * 设定物理内存为 64MB，并据此计算 npage。*/
25  // 设定 64MB 物理内存，计算页数 npage，并打印内存信息
26  void mips_detect_memory() // 中文注释
27  { // 中文注释
28      /* 步骤 1：初始化 basemem。
29       * 在真实硬件上可由 CMOS 提供容量信息。 */
30      maxpa = 0x4000000; // 中文注释
31      basemem = 0x4000000; // 中文注释
32      extmem = 0; // 中文注释
33      npage = maxpa/4096; // 中文注释
34      // 步骤 2：计算对应的 npage 值。
35
36      printf("Physical memory: %dK available, ", (int)(maxpa /
37          ↪ 1024)); // 中文注释
38      printf("base = %dK, extended = %dK\n", (int)(basemem / 1024),
39          ↪ // 中文注释
40          (int)(extmem / 1024)); // 中文注释
41  } // 中文注释
```

2. mips_vm_init()

mm/pmap.c (132-164)

```

132  /* 概览：建立二级页表。
133  * 提示：UPAGES 与 UENVS 的布局参见 include/mmu.h。*/
134  // 分配页目录与管理数组，并将 pages/envs 映射到固定高地址区
135  void mips_vm_init() // 中文注释
136  { // 中文注释
137      extern char end[]; // 中文注释
138      extern int mCONTEXT; // 中文注释
139      extern struct Env *envs; // 中文注释
140
141      Pde *pgdir; // 中文注释
142      u_int n; // 中文注释
143
144      /* 步骤 1：为一级页表（页目录）分配一页 */
145      pgdir = alloc(BY2PG, BY2PG, 1); // 中文注释
146      printf("to memory %x for struct page directory.\n", freemem);
147      ↪ // 中文注释
148      mCONTEXT = (int)pgdir; // 中文注释
149
150      boot_pgdir = pgdir; // 中文注释
151
152      /* 步骤 2：为全局数组 pages 分配物理内存并映射到 UPAGES；
153      * 注意在映射前将大小按页上取整以满足对齐。 */
154      pages = (struct Page *)alloc(npage * sizeof(struct Page),
155      ↪ BY2PG, 1); // 中文注释
156      printf("to memory %x for struct Pages.\n", freemem); //
157      ↪ 中文注释
158      n = ROUND(npage * sizeof(struct Page), BY2PG); // 中文注释
159      boot_map_segment(pgdir, UPAGES, n, PADDR(pages), PTE_R); //
160      ↪ 中文注释
161
162      /* 步骤 3：为进程管理的全局数组 envs 分配物理内存并映射到 UENVS。 */
163      envs = (struct Env *)alloc(NENV * sizeof(struct Env), BY2PG,
164      ↪ 1); // 中文注释

```

```

160     n = ROUND(NENV * sizeof(struct Env), BY2PG); // 中文注释
161     boot_map_segment(pgdir, UENVS, n, PADDR(envs), PTE_R); //
        ↪ 中文注释
162
163     printf("pmap.c:\t mips vm init success\n"); // 中文注释
164 } // 中文注释

```

3. page_init()

mm/pmap.c (166-191)

```

166 /* 概览：初始化 Page 结构与空闲链表。
167  * 每个物理页对应 pages 数组中的一个 struct Page；页有引用计数，
        ↪ 空闲页挂在链表上。 // 中文注释
168  * 提示：用 LIST_INSERT_HEAD 插入链表。*/
169 // 初始化空闲页链表，将 freemem 以下视为已占用，其余页挂入空闲链表
170 void // 中文注释
171 page_init(void) // 中文注释
172 { // 中文注释
173     int count = 0; // 中文注释
174     /* 步骤 1：初始化 page_free_list (参见 include/queue.h 的
        ↪ LIST_INIT) */
175     LIST_INIT(&page_free_list); // 中文注释
176
177     /* 步骤 2：将 freemem 向上对齐到 BY2PG 的整数倍 */
178     freemem = ROUND(freemem, BY2PG); // 中文注释
179
180     /* 步骤 3：将 freemem 以下的页标记为已用 (pp_ref 设为 1) */
181     //LIST_FOREACH(pages, page_free_list, pp_link) {
182     for (count = 0; count < PADDR(freemem)/BY2PG; count++) { //
        ↪ 中文注释
183         pages[count].pp_ref = 1; // 中文注释
184     } // 中文注释
185
186     /* 步骤 4：其余页标记为空闲并挂入空闲链表 */
187     for (count; count < npage; count++) { // 中文注释
188         pages[count].pp_ref = 0; // 中文注释

```

```
189     LIST_INSERT_HEAD(&page_free_list, &pages[count], pp_link); //
        ↪ 中文注释
190     } // 中文注释
191 } // 中文注释
```

五、内存分配与释放

1. page_alloc() 主体

mm/pmap.c (193-215)

```
193 /* 概览：从空闲内存分配一页，并清零。
194  * 事后条件：若无空闲页返回 -E_NO_MEM；否则将获取的页地址写入 *pp 并返回 0。
        ↪ // 中文注释
195  * 说明：不在此处增加引用计数，必要时由调用方处理（或通过 page_insert）。 //
        ↪ 中文注释
196  * 提示：使用 LIST_FIRST 与 LIST_REMOVE 操作链表。*/
197 // 从空闲链表分配一页（清零），空则返回 -E_NO_MEM
198 int // 中文注释
199 page_alloc(struct Page **pp) // 中文注释
200 { // 中文注释
201     struct Page *ppage_temp; // 中文注释
202
203     /* 步骤 1：从空闲链表获取一页；若失败返回错误码 */
204     if (LIST_EMPTY(&page_free_list)) { // 中文注释
205         return -E_NO_MEM; // 中文注释
206     } // 中文注释
207
208     ppage_temp = LIST_FIRST(&page_free_list); // 中文注释
209     LIST_REMOVE(ppage_temp, pp_link); // 中文注释
210
211     /* 步骤 2：初始化该页（使用 bzero 清零） */
212     *pp = ppage_temp; // 中文注释
213     bzero((void *)page2kva(ppage_temp), BY2PG); // 中文注释
214
215     return 0; // 中文注释
```

BFS 层 1: 直接依赖的辅助逻辑

- LIST_EMPTY: 判空空闲链表, 决定是否返回 -E_NO_MEM。实现见 queue.h 第 18 行。

include/queue.h (18-18)

```
18 #define LIST_EMPTY(head)      ((head)->lh_first == NULL) //
    ↪ 判空
```

- LIST_FIRST: 取得链表首元指针供分配使用。

include/queue.h (19-19)

```
19 #define LIST_FIRST(head)      ((head)->lh_first)          //
    ↪ 取首元
```

- LIST_REMOVE: 将首元从空闲链摘除, 确保不会被重复分配。

include/queue.h (71-76)

```
71 #define LIST_REMOVE(elm, field) do {
    ↪ \
72     if (LIST_NEXT((elm), field) != NULL)
    ↪ \
73         LIST_NEXT((elm),
    ↪ field)->field.le_prev =
    ↪ \
74             (elm)->field.le_prev;
    ↪ \
75     *(elm)->field.le_prev = LIST_NEXT((elm),
    ↪ field);
    ↪ \
76 } while (0) // 删除指定元素
```

- page2kva: 把 struct Page 转为可写的内核虚拟地址, 以便 bzero 清空实际页帧内容。

include/pmap.h (34-36)

```

34 // Page* -> 可访问的内核虚拟地址 (通过 KADDR 恒等映射)
35 static inline u_long page2kva(struct Page* pp) { return
    ↪ KADDR(page2pa(pp)); }
36

```

- bzero: 逐字写零的公共例程, 用于满足实验要求的“返回前清零”。

init/init.c (67-89)

```

67 void bzero(void *b, size_t len)           // 内存清零:
    ↪ 将起始地址 b 开始的 len 字节全部置 0
68 { // 函数体开始
69     void *max;                            // 清零范围末地址
    ↪ (开区间)
70
71     max = b + len;                        // 计算末尾地址,
    ↪ 便于边界判断
72
73     //printf("init.c:\tzzero from %x to
    ↪ %x\n", (int)b, (int)max); // 调试输出: 观察清零范围
    ↪ (可按需启用)
74
75     // 尽可能按机器字 (4 字节) 对齐清零, 提高效率
76
77     while (b + 3 < max)
78     {
79         *(int *)b = 0;                    // 写入 4 字节 0
80         b+=4;                             // 指针前移 4
            ↪ 字节
81     }
82
83     // 处理剩余的 0~3 个零散字节, 逐字节清零补齐
84     while (b < max)
85     {

```

```

86     *(char *)b++ = 0;           // 写入 1 字节 0
      ↪ 并前移
87 }
88
89 } // 函数体结束

```

分配流程小结 依次执行“判空 → 弹出链表 → 写入返回指针 → 清零物理页”，且不在此处修改 `pp_ref`，把引用计数的职责交给后续 `page_insert` 或调用者，满足原分析报告对“`page_alloc` 清零但不自增 `pp_ref`”的要求。

2. `page_free()` 主体

mm/pmap.c (218-236)

```

218 /* 概览：释放一页；当 pp_ref 递减至 0 时将其标记为空闲。
219  * 提示：释放时将该页插入 page_free_list。*/
220 // 当 pp_ref==0 时归还；==1 表示仍被引用，直接返回
221 void // 中文注释
222 page_free(struct Page *pp) // 中文注释
223 { // 中文注释
224     /* 步骤 1：若仍有虚拟地址引用该页，则不处理 */
225     if (pp->pp_ref == 1) { // 中文注释
226         return; // 中文注释
227     } // 中文注释
228
229     /* 步骤 2：当 pp_ref 为 0 时，标记为空闲并返回 */
230     if (pp->pp_ref == 0) { // 中文注释
231         LIST_INSERT_HEAD(&page_free_list, pp, pp_link); // 中文注释
232         return; // 中文注释
233     } // 中文注释
234     /* 若 pp_ref 小于 0，说明早前出现错误，直接 panic */
235     panic("cgh:pp->pp_ref is less than zero\n"); // 中文注释
236 } // 中文注释

```

BFS 层 1：直接依赖

- `LIST_INSERT_HEAD`：当 `pp_ref == 0` 时，把页面重新挂到空闲链表头部。

include/queue.h (48-53)

```

48 #define LIST_INSERT_HEAD(head, elm, field) do {
    ↪ \
49     if ((LIST_NEXT((elm), field) =
    ↪ LIST_FIRST((head))) != NULL) \
50         LIST_FIRST((head))->field.le_prev =
    ↪ &LIST_NEXT((elm), field); \
51     LIST_FIRST((head)) = (elm);
    ↪ \
52     (elm)->field.le_prev = &LIST_FIRST((head));
    ↪ \
53 } while (0) // 头插

```

- panic: 若 `pp_ref < 0`, 认为发生了双重释放或越界操作, 立即停止系统。

include/printf.h (7-10)

```

7 void _panic(const char*, int, const char*, ...) //
    ↪ 内核致命错误输出并停机
8     __attribute__((noreturn)); // 函数不返回
9
10 #define panic(...) _panic(__FILE__, __LINE__, __VA_ARGS__) //
    ↪ 带文件与行号的 panic 宏

```

回收流程小结 函数首先识别“仍被引用”的页 (`pp_ref == 1`) 并保持不变; 仅当引用计数降到 0 时才回收到 `page_free_list`。出现负值会触发 `panic`, 迫使调试者回溯引用计数逻辑。

整体内存管理总结 结合 `page_alloc/page_free` 可见, 物理内存生命周期严格依赖 `pp_ref`: 所有可分配页都集中在 `page_free_list`, 每次分配都需要合作的 `page_insert`、`page_remove` 在建立/拆除映射时维护引用计数。这种设计满足原分析报告中“维护 `pp_ref` 与 `tlb_invalidate` 协同”的提示。

六、地址转换与 TLB 重填

1. 地址转换方法概述

MIPS 两级页表配合自映射窗口的转换流程如下：

1. 通过 $PDX(va)/PTX(va)$ 划分虚拟地址的 10+10+12 位；2. $va2pa()$ 先用 PDX 找到页目录项，若 PTE_V 未置位直接返回 ~ 0 ；3. 若页目录有效，则将其物理地址经 $KADDR$ 映射成内核虚拟地址获取二级页表；4. 再检查对应 PTE 的 PTE_V ，最终返回 PTE_ADDR 所给出的物理页基址。

上述逻辑直接体现在 `include/pmap.h`：

`include/pmap.h (38-47)`

```

38 static inline u_long va2pa(Pde* pgdir, u_long va) {
39     Pte* p;
40     pgdir = &pgdir[PDX(va)];
41     if (!(*pgdir & PTE_V))
42         return ~0;
43     p = (Pte*)KADDR(PTE_ADDR(*pgdir));
44     if (!(p[PTX(va)] & PTE_V))
45         return ~0;
46     return PTE_ADDR(p[PTX(va)]);
47 }
```

2. TLB 重填流程（在原分析报告原文基础上润色）

当使用 `kuseg` 虚拟地址访问内存而 TLB 未命中时，CPU 触发 `handle_tlb`，随后跳到 `lib/genex.S` 中的 `do_refill`。其工作步骤为：“保存上下文 → 读取 `CP0_BADVADDR/ENTRYHI` → 检查 V 位 → 若缺页则调用 `pageout` → 成功时写 `EntryLo` → `tlbwr` 写随机项”。核心代码如下：

`lib/genex.S (80-148)`

```

80     .extern tlbwa
81     .set     noreorder
82     NESTED(do_refill, 0, sp)
83         //li     k1, '?'
84         //sb     k1, 0x90000000
```

```
85 |         .extern      mCONTEXT
86 | //this "1" is important
87 | 1:         //j 1b
88 |         nop
89 |         lw          k1,mCONTEXT
90 |         and         k1,0xffffffff000
91 |         mfc0        k0,CP0_BADVADDR
92 |         srl         k0,20
93 |         and         k0,0xfffffffffc
94 |         addu        k0,k1
95 |
96 |         lw          k1,0(k0)
97 |         nop
98 |         move        t0,k1
99 |         and         t0,0x0200
100 |        beqz         t0,NOPAGE
101 |        nop
102 |        and         k1,0xffffffff000
103 |        mfc0        k0,CP0_BADVADDR
104 |        srl         k0,10
105 |        and         k0,0xfffffffffc
106 |        and         k0,0x00000fff
107 |        addu        k0,k1
108 |
109 |        or          k0,0x80000000
110 |        lw          k1,0(k0)
111 |        nop
112 |        move        t0,k1
113 |        and         t0,0x0200
114 |        beqz         t0,NOPAGE
115 |        nop
116 |        move        k0,k1
117 |        and         k0,0x1
118 |        beqz         k0,NoCOW
119 |        nop
120 |        and         k1,0xfffffbff
```

```

121 NoCOW:
122         mtc0          k1,CP0_ENTRYLO0
123         nop
124         tlbwr
125
126         j             2f
127         nop
128 NOPAGE:
129 //3: j 3b
130 nop // 中文注释: 延迟槽
131         mfc0          a0,CP0_BADVADDR // 中文注释: a0 =
           ↳ 故障地址
132         lw           a1,mCONTEXT // 中文注释: a1 = 地址空间上下文
133         nop // 中文注释: 延迟槽
134
135         sw           ra,tlbra
136         jal         pageout
137         nop
138 //3: j 3b
139 nop // 中文注释: 占位 (调试分支去除)
140         lw           ra,tlbra // 中文注释: 恢复返回地址
141         nop // 中文注释: 延迟槽
142
143         j           1b // 中文注释: 回到重试点
144 2:      nop // 中文注释: 延迟槽
145
146         jr          ra // 中文注释: 返回上一层
147         nop // 中文注释: 延迟槽
148 END(do_refill) // 中文注释: TLB 重填处理结束

```

结合源码可对原分析报告文本进行更精确表述:

- mfc0 a0, CP0_BADVADDR 与 mfc0 a1, CP0_ENTRYHI 把故障虚拟地址与 ASID 作为 C 例程参数;
- sw ra, tlbra 保存返回地址, 确保调用 pageout 或其他 C 例程后能回到汇编流程;
- 若页表项的 PTE_V 置位, 直接 mtc0 k1, CP0_ENTRYLO0 并执行 tlbwr;
- 否则跳转到 NOPAGE 分支, 调用 pageout 触发懒分配, 再回到标签 1b 重试, 直到页表项有效。

[illegible]

七、附加题：建表与映射函数

原分析报告末尾的三个题目与五星等价：均要求“逐语句注释 + 调用链 BFS 分析”。以下分别给出 `boot_pgdir_walk`、`pgdir_walk`、`boot_map_segment` 的代码引用与调用图解。

1. `boot_pgdir_walk`：启动阶段页表分配

mm/pmap.c (80-106)

```

80  /* 概览：在给定的页目录 pgdir 中，获取虚拟地址 va 对应的页表项指针。
81  * 若不存在相应页表且 create==1，则按需创建该页表。*/
82  // 启动阶段定位/按需创建二级页表，返回 VA 对应的 PTE 指针
83  static Pte *boot_pgdir_walk(Pde *pgdir, u_long va, int create) //
    ↪ 中文注释
84  { // 中文注释
85      //pgdir_entryp 是页目录记录的二级页表 pgtable 的物理地址
86      Pde *pgdir_entryp; // 中文注释
87      Pte *pgtable, *pgtable_entry; // 中文注释
88
89      /* 步骤 1：取得对应的页目录项与页表。
90       * 提示：借助 KADDR 和 PTE_ADDR 从目录项值得到页表地址。 */
91      pgdir_entryp = &pgdir[PDX(va)]; // 中文注释
92      pgtable = (Pte *)KADDR(PTE_ADDR(*pgdir_entryp)); //将物理地址对应到内核虚拟地址
93
94      /* 步骤 2：若页表不存在且 create==1，则创建之，并设置合适的权限位。 */
95      if (!(*pgdir_entryp & PTE_V) && create == 1) { // 中文注释
96          pgtable = (Pte *)alloc(BY2PG, BY2PG, 1); // 中文注释
97          *pgdir_entryp = PTE_ADDR(PADDR(pgtable)) | PTE_V; // | PTE_R;
98      } // 中文注释
99      else if (!(*pgdir_entryp & PTE_V) && create == 0) { // 中文注释
100         return 0; // 中文注释
101     } // 中文注释
102
103     /* 步骤 3：返回 va 对应的页表项指针 */
104     pgtable_entry = &pgtable[PTX(va)]; // 中文注释
105     return pgtable_entry; // 中文注释

```

```
106 } // 中文注释
```

BFS 层 1: 直接依赖

- PDX: 定位目录项索引。

include/mmu.h (11-11)

```
11 #define PDX(va) (((u_long)(va)) >> 22) & 0x03FF) //
    ↪ 页目录索引 (高 10 位)
```

- KADDR + PTE_ADDR: 若目录项已有页表, 则组合使用把物理地址转为内核线性地址并得到指向页表的指针。

include/mmu.h (91-97)

```
91 #define KADDR(pa)                                \
92 ({                                              \
93     u_long ppn = PPN(pa);                      \
94     if (ppn >= npage)                          \
95         panic("KADDR called with invalid pa %08lx", \
96             ↪ (u_long)pa); \
97     (pa) + ULIM;                                \
97 })
```

include/mmu.h (14-14)

```
14 #define PTE_ADDR(pte) ((u_long)(pte) & ~0xFFF) // PTE
    ↪ 对应的物理页框基址
```

- alloc(): 当页表不存在且 create==1 时, 调用线性分配器按页对齐分配一整张页表。

mm/pmap.c (41-78)

```
41  /* 概览：分配对齐的物理内存块（大小 n，按 align 对齐）；若 clear
    ↳ 置位则清零。
42  * 该分配器仅用于虚拟内存系统建立阶段。 // 中文注释
43  * // 中文注释
44  * 事后条件：若内存不足则触发 panic，否则返回已分配内存的地址。*/
45  // 启动期线性分配器：按对齐从 freemem 线性划拨；可选清零；越界则 panic
46  static void *alloc(u_int n, u_int align, int clear) // 中文注释
47  { // 中文注释
48      extern char end[]; // 中文注释
49      u_long allocated_mem; // 中文注释
50
51      /* 首次使用时初始化 freemem:
    ↳ 取链接器未分配给内核代码/全局变量的首个虚拟地址。 */
52      if (freemem == 0) { // 中文注释
53          freemem = (u_long)end; // 中文注释
54      } // 中文注释
55
56      /* 步骤 1：将 freemem 向上按 align 对齐 */
57      freemem = ROUND(freemem, align); // 中文注释
58
59      /* 步骤 2：保存当前 freemem 作为已分配块起始 */
60      allocated_mem = freemem; // 中文注释
61
62      /* 步骤 3：前移 freemem 记录分配 */
63      freemem = freemem + n; // 中文注释
64
65      /* 步骤 4：若 clear 为真，则清零已分配区域 */
66      if (clear) { // 中文注释
67          bzero((void *)allocated_mem, n); // 中文注释
68      } // 中文注释
69
70      // 内存不足，触发 panic
71      if (PADDR(freemem) >= maxpa) { // 中文注释
72          panic("out of memory\n"); // 中文注释
73          return (void *)-E_NO_MEM; // 中文注释
74      } // 中文注释
```

```

75 |
76 |     /* 步骤 5: 返回已分配块地址 */
77 |     return (void *)allocated_mem; // 中文注释
78 | } // 中文注释

```

- PADDR: 把新页表的内核虚拟地址翻译成物理地址后写回目录项。

include/mmu.h (82-88)

```

82 | #define PADDR(kva) \
83 | ({ \
84 |     u_long a = (u_long)(kva); \
85 |     if (a < ULIM) \
86 |         panic("PADDR called with invalid kva %08lx", a); \
87 |     a - ULIM; \
88 | })

```

流程总结 函数先读目录项，如果 PTE_V 清零且允许创建，就分配一页、清零并写入 PTE_V；若不允许创建则返回 NULL。否则最终返回 va 对应的二级页表项指针，供 boot_map_segment 写入映射。

2. pgdir_walk: 运行期页表定位

mm/pmap.c (238-275)

```

238 | /* 概览: 在页目录 pgdir 中定位虚拟地址 va 的页表项 (权限 PTE_R|PTE_V)。
239 | * 前置条件: pgdir 为二级页表结构。 // 中文注释
240 | * 事后条件: 若内存不足返回 -E_NO_MEM; 否则返回 0 并通过 *ppte
    |   ↪ 给出页表项指针。 // 中文注释
241 | * 提示: 使用二级指针返回页表项指针; 与 boot_pgdir_walk 类似。*/
242 | // 运行期 PTE 定位, 必要时按需分配二级页表
243 | int // 中文注释
244 | pgdir_walk(Pde *pgdir, u_long va, int create, Pte **ppte) //
    |   ↪ 中文注释
245 | { // 中文注释
246 |     Pde *pgdir_entryp; // 中文注释
247 |     Pte *pgtable; // 中文注释

```

```

248     struct Page *ppage; // 中文注释
249     int err_num; // 中文注释
250
251     /* 步骤 1: 取得对应的页目录项与页表 */
252     pgdir_entryp = &pgdir[PDX(va)]; // 中文注释
253     pgtable = (Pte *)KADDR(PTE_ADDR(*pgdir_entryp)); // 中文注释
254
255     /* 步骤 2: 若页表不存在且 create==1, 则创建; 并设置合适权限位
        ↳ (可能内存不足)。 */
256     if (!(*pgdir_entryp & PTE_V) && create == 1) { // 中文注释
257         err_num = page_alloc(&ppage); // 中文注释
258         if (err_num != 0) { // 中文注释
259             return err_num; // 中文注释
260         } // 中文注释
261         ppage->pp_ref++; // 中文注释
262         pgtable = (Pte *)page2kva(ppage); // 中文注释
263         /*pgdir_entry = PTE_ADDR(PADDR(pgtable));
264         *pgdir_entryp = page2pa(ppage) | PTE_V; // | PTE_R;
265         */ // 中文注释
266     else if (!(*pgdir_entryp & PTE_V) && create == 0) { // 中文注释
267         *ppte = 0; // 中文注释
268         return 0; // 中文注释
269     } // 中文注释
270
271     /* 步骤 3: 将页表项地址写入 *ppte 作为返回值 */
272     *ppte = &pgtable[PTX(va)]; // 中文注释
273
274     return 0; // 中文注释
275 } // 中文注释

```

BFS 层 1: 直接依赖

- PDX/KADDR/PTE_ADDR: 与启动版本一致, 用于找到已有页表基址。

```
include/mmu.h (11-14)
```

```

11 #define PDX(va) (((u_long)(va)) >> 22) & 0x03FF) //
    ↪ 页目录索引 (高 10 位)
12 #define PTX(va) (((u_long)(va)) >> 12) & 0x03FF) // 页表索引
    ↪ (中 10 位)
13
14 #define PTE_ADDR(pte) ((u_long)(pte) & ~0xFFF) // PTE
    ↪ 对应的物理页框基址

```

include/mmu.h (91-97)

```

91 #define KADDR(pa)                                \
92 ({                                              \
93     u_long ppn = PPN(pa);                      \
94     if (ppn >= npage)                          \
95         panic("KADDR called with invalid pa %08lx", \
96             ↪ (u_long)pa); \
97     (pa) + ULIM;                                \
98 })

```

- `page_alloc`: 若需要新页表, 调用运行期分配器获取一个 `struct Page`。

mm/pmap.c (193-215)

```

193 /* 概览: 从空闲内存分配一页, 并清零。
194  * 事后条件: 若无空闲页返回 -E_NO_MEM; 否则将获取的页地址写入 *pp
195  * 说明: 不在此处增加引用计数, 必要时由调用方处理 (或通过 page_insert)。
196  * 提示: 使用 LIST_FIRST 与 LIST_REMOVE 操作链表。*/
197 // 从空闲链表分配一页 (清零), 空则返回 -E_NO_MEM
198 int // 中文注释
199 page_alloc(struct Page **pp) // 中文注释
200 { // 中文注释
201     struct Page *ppage_temp; // 中文注释
202

```

```

203     /* 步骤 1: 从空闲链表获取一页; 若失败返回错误码 */
204     if (LIST_EMPTY(&page_free_list)) { // 中文注释
205         return -E_NO_MEM; // 中文注释
206     } // 中文注释
207
208     ppage_temp = LIST_FIRST(&page_free_list); // 中文注释
209     LIST_REMOVE(ppage_temp, pp_link); // 中文注释
210
211     /* 步骤 2: 初始化该页 (使用 bzero 清零) */
212     *pp = ppage_temp; // 中文注释
213     bzero((void *)page2kva(ppage_temp), BY2PG); // 中文注释
214
215     return 0; // 中文注释

```

- page2kva/page2pa: 分别把 struct Page 转成内核虚拟地址 (写 PTE 用) 与物理地址 (写 PDE 用)。

include/pmap.h (24-36)

```

24 // Page* -> 物理地址 (页号左移 12 得到页框基址)
25 static inline u_long page2pa(struct Page* pp) { return
    ↪ page2ppn(pp) << PGSHIFT; }
26
27 // 物理地址 -> Page* (越界检查: PPN(pa) 必须小于 npage)
28 static inline struct Page* pa2page(u_long pa) {
29     if (PPN(pa) >= npage)
30         panic("pa2page called with invalid pa: %x", pa);
31     return &pages[PPN(pa)];
32 }
33
34 // Page* -> 可访问的内核虚拟地址 (通过 KADDR 恒等映射)
35 static inline u_long page2kva(struct Page* pp) { return
    ↪ KADDR(page2pa(pp)); }
36

```

- pp_ref++: 虽然不是函数, 但表明页表页本身也需要引用计数管理, 避免提前回

收。

流程总结 `pgdir_walk` 在运行期负责 lazy allocation: 允许创建时若目录项无效, 则分配新页表、递增引用计数并写入 `PTE_V`; 如 `create==0` 则返回 0 表示“不存在”。最终把 `va` 对应的二级 PTE 指针存入 `*ppte` 供 `page_insert` 等调用者继续操作。

3. `boot_map_segment`: 批量建立映射

`mm/pmap.c (108-130)`

```

108  /* 概览: 将虚拟区间 [va, va+size) 映射到物理区间 [pa, pa+size), 根为
    ↪ pgdir。
109  * 条目权限为 perm|PTE_V。 // 中文注释
110  * // 中文注释
111  * 前置条件: size 必须是 BY2PG 的整数倍。*/
112  // 区间映射: [va,va+size) → [pa,pa+size), 逐页写 PTE (perm|PTE_V)
113  void boot_map_segment(Pde *pgdir, u_long va, u_long size, u_long
    ↪ pa, int perm) // 中文注释
114  { // 中文注释
115      int i, va_temp; // 中文注释
116      Pte *pgtable_entry; // 中文注释
117
118      /* 步骤 1: 检查 size 是否是 BY2PG 的整数倍 */
119      if (size % BY2PG != 0) { // 中文注释
120          return; // 中文注释
121      } // 中文注释
122
123      /* 步骤 2: 执行区间映射 */
124      /* 提示: 用 boot_pgdir_walk 获取 va 的页表项 */
125      for (va_temp = va; va_temp < va + size; va_temp += BY2PG, pa
    ↪ += BY2PG) { // 中文注释
126          pgtable_entry = boot_pgdir_walk(pgdir, va_temp, 1); //
    ↪ 中文注释
127          *pgtable_entry = pa | perm | PTE_V; // 中文注释
128      } // 中文注释
129
130  } // 中文注释

```

BFS 层 1: 直接依赖

- BY2PG: 循环增量与合法性检查均以页大小为单位, 确保 `size` 为整页数。

include/mmu.h (5-5)

```
5 #define BY2PG    4096           // 每页 4KB
```

- `boot_pgdir_walk`: 对区间内的每一页调用一次, 按需创建 PTE。

mm/pmap.c (80-106)

```
80 /* 概览: 在给定的页目录 pgdir 中, 获取虚拟地址 va 对应的页表项指针。
81  * 若不存在相应页表且 create==1, 则按需创建该页表。*/
82 // 启动阶段定位/按需创建二级页表, 返回 VA 对应的 PTE 指针
83 static Pte *boot_pgdir_walk(Pde *pgdir, u_long va, int
   ↳ create) // 中文注释
84 { // 中文注释
85     //pgdir_entryp 是页目录记录的二级页表 pgtable 的物理地址
86     Pde *pgdir_entryp; // 中文注释
87     Pte *pgtable, *pgtable_entry; // 中文注释
88
89     /* 步骤 1: 取得对应的页目录项与页表。
90      * 提示: 借助 KADDR 和 PTE_ADDR 从目录项值得到页表地址。 */
91     pgdir_entryp = &pgdir[PDX(va)]; // 中文注释
92     pgtable = (Pte *)KADDR(PTE_ADDR(*pgdir_entryp));
   ↳ //将物理地址对应到内核虚拟地址
93
94     /* 步骤 2: 若页表不存在且 create==1, 则创建之, 并设置合适的权限位。
   ↳ */
95     if (!(*pgdir_entryp & PTE_V) && create == 1) { // 中文注释
96         pgtable = (Pte *)alloc(BY2PG, BY2PG, 1); // 中文注释
97         *pgdir_entryp = PTE_ADDR(PADDR(pgtable)) | PTE_V; // |
   ↳ PTE_R;
98     } // 中文注释
99     else if (!(*pgdir_entryp & PTE_V) && create == 0) { //
   ↳ 中文注释
```

```
100     return 0; // 中文注释
101 } // 中文注释
102
103 /* 步骤 3: 返回 va 对应的页表项指针 */
104 pgtable_entry = &pgtable[PTX(va)]; // 中文注释
105 return pgtable_entry; // 中文注释
106 } // 中文注释
```

- PTE_V 与 perm: 每次写入 PTE 时组合物理地址与权限位, 保证 1:1 包含 PTE_V。

include/mmu.h (25-27)

```
25 #define PTE_G      0x0100 // 全局页
26 #define PTE_V      0x0200 // 有效
27 #define PTE_R      0x0400 // 可写/脏 (0 表示只读)
```

流程总结 函数先确认 `size` 是否整页, 否则直接返回; 随后以 `BY2PG` 为步长遍历目标区间, 对每个虚拟地址调用 `boot_pgdir_walk(pgdir, va_temp, 1)` 保证存在 PTE, 再把当前 `pa` 与权限写入条目。该流程在 `mips_vm_init()` 中被调用两次, 构建 UPAGES/UENVS 的只读窗口。